



Toolset editor

Coordinate MCP toolset editing through an explicit host adapter.

Problem and boundary

Public contract

Working integration

@epam/ai-dial-toolset-editor



When this library is useful

Coordinate MCP toolset editing through an explicit host adapter.

01

Create a toolset

Edit metadata, endpoint, transport and authentication settings.

02

Persist a draft

Reuse the identifier returned by the first successful save.

03

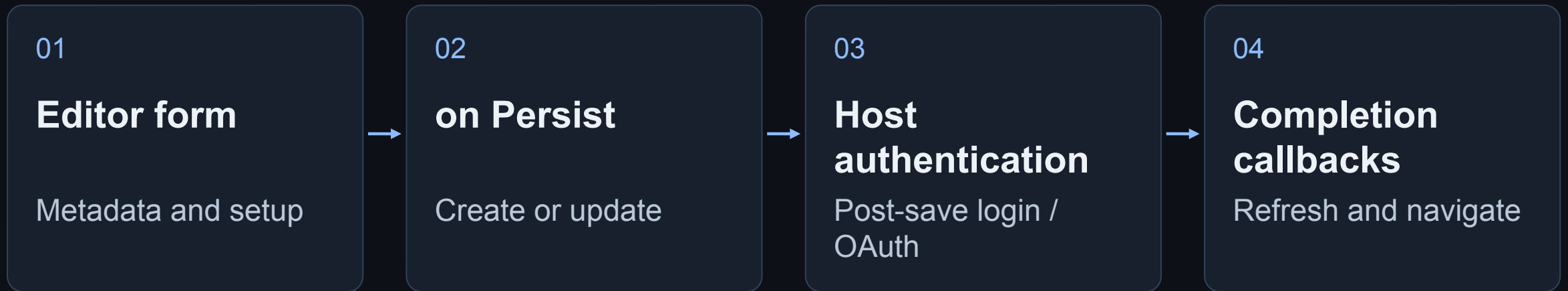
Authorize access

Delegate login and OAuth to host behavior.



Follow the ownership boundary

Inputs and effects stay explicit.



Library: Form state, dirty/errors state and save orchestration around injected operations.



The public contract to start with

ToolsetEditor · selected surface; full API index is in the bundle.

Input or operation	Contract
initialForm	ToolsetFormData
onPersist	(form: ToolsetFormData, toolsetId: string) => Promise<string null>
onOAuthLogin	ToolsetOAuthLoginHandler
authActions	ToolsetAuthActions



A minimal workspace integration

TypeScript / React example · imports and host setup abridged.

```
function ToolsetEditorDemo(host: ToolsetEditorProps) {  
  return <ToolsetEditor {...host} />;  
}  
  
// host includes required persistence, authentication,  
// notification, file-manager and navigation callbacks.  
// See the complete contract in content/api-index.md.
```

Full example: [examples/toolset-editor.tsx](#) · uses [@epam/ai-dial-toolset-editor](#)



Wire a realistic host workflow

Keep toolset editor focused on its public responsibility.

01

Prepare the adapter

Supply the required persistence, file and auth callbacks.

02

Collect toolset values

Let the editor coordinate metadata and setup controls.

03

Run host operations

Keep credential flows, endpoint calls and routing in the host.



Validate a candidate endpoint

Use an exported utility before host persistence.

```
const valid = isValidEndpointUrl(  
  'https://example.com/mcp',  
);
```

Focused source-backed example · see [examples/](#) and [content/api-index.md](#).



Available customization

01

Typed integration

`ToolsetEditorProps` makes persistence, auth and file dependencies explicit.

02

Shared metadata

`GeneralForm` reuses builder-form fields and validation.

03

MCP URL resolver

`buildMcpUrl` receives the current draft identifier.



Find the integration and its checks

Existing tests are evidence of intended behavior; suite results are not claimed.

Evidence	Where to look
Real consumer	chat/pages/ToolsetEditor/ToolsetEditor.tsx
Test evidence	renders the default title, description, and copy button
Test evidence	calls onAuthChange with ApiKey type when the ApiKey option is clicked



Constraints that affect integration

01

No minimal fake backend

A working editor needs the required host callbacks and file component.

02

Form identity

A new `initialForm` object reseeds the editing session.

03

Boundary discrepancy

Current code imports chat-hooks symbols; document this wider dependency.